

Replottable, Not Rerunnable

What a Figure Repository Lets a Reader Check

Flyxion
Independent Researcher

September 2026

Abstract

The spintronic Ising machine paper in Nature Electronics is accompanied by a public repository containing source data and plotting notebooks for three of its figures. The machine code and fabrication process are not included, and code and problem instances are available only on request. Such a release is usually described as supporting figure regeneration and nothing more. This essay distinguishes four capacities a release can support: replotting a figure from deposited records, verifying a claim about those records against an independent criterion, rerunning the experiment that produced them, and rebuilding a comparison across systems. The first three form a chain; the fourth is a separate axis. The repository was meant to support the first. Because its notebooks define the problem instances, the Hamiltonians, and the metrics, it also supports the second, and exercising it produces three findings. The dashed lines labelled “Ground state” in Figs. 4d and 5d are computed as the minimum energy along each plotted trajectory. For the routing instance defined in the notebook, the tree shown as optimal has length 36, while exhaustive enumeration finds a valid tree of length 32 that scores lower under the notebook’s own Hamiltonian. And the optimality gap in Fig. 5f is measured against the best of 100 trials at each iteration count, a reference that makes it a measure of dispersion rather than of distance from the optimum. None of these checks requires the chip. They are possible only because something was released, and they cannot be resolved without what was not.

1 What Was Released

The paper’s data availability statement points to a GitHub repository for source data, with other data available on reasonable request, and its code availability statement makes the computer code and problem instances available from the corresponding author on reasonable request [1]. The repository contains three Jupyter notebooks, one each for Figs. 2, 4, and 5; a data directory with the arrays those notebooks read; the rendered figure panels as SVG files; and an MIT license. Its README states that it “contains source data and plotting scripts used to generate figures,” and that “device fabrication process and Ising machine codes are not included” [2].

The data are specific. For Fig. 2, there are switching probabilities against magnetic field and pulse width, and resistance traces over consecutive pulses. For Fig. 4, there is the spin configuration of the routing instance at each of 1,000 annealing iterations, and a spreadsheet of outcome frequencies across penalty settings and iteration counts. For Fig. 5, there are the coupling matrix J and fields h of a 40-segment layer-assignment instance, two spin trajectories of 1,000 iterations each, the segment geometry, and final-energy arrays for problems of 20 to 100 segments.

A release of this kind is common, and it is better than none. The question is what it allows a reader to do.

2 Four Capacities

Definition 1 (Capacities of a release). A release supports

1. replotting a figure if the deposited records and code reproduce the figure;
2. verifying a claim if it contains enough of the problem definition and the claimed outcome for a reader to check the claim against a criterion independent of the authors’ computation;
3. rerunning an experiment if it contains enough to regenerate the records themselves: instances, mapping, schedule, update rule, and either the hardware or a model of it;
4. rebuilding a comparison if it contains the per-system quantities, their provenance, and the procedure by which they were made commensurable.

The first three form a chain. Rerunning requires what verifying requires, and verifying requires the records that replotting uses. The fourth does not sit on the chain. Rebuilding the paper’s comparison table would require the per-platform time-to-solution figures and the estimation procedure for rows that were never measured on the benchmark, as the companion essay on the table describes [6]; it would not require rerunning the machine. Conversely, rerunning the machine would not rebuild the table. A release can be strong on one axis and empty on the other.

These capacities map onto the vocabulary used by the National Academies, in which reproducibility means obtaining consistent results from the same data and code, and replicability means obtaining consistent results from new data [3]. Replotting is a narrow form of reproducibility that stops at the rendering step. Verification is not usually named at all, and it is the capacity this essay is concerned with, because it is the one that turns out to be available here almost by accident.

3 What the Notebooks Contain

A plotting notebook is often thought of as cosmetic: axis labels, colors, font sizes. These notebooks contain more. The Fig. 4 notebook defines the routing instance explicitly, as seven terminal coordinates and eight candidate Steiner points, builds the distance and rectilinearity matrices, and implements the full routing Hamiltonian with its penalty weights ($\lambda_1 = \lambda_2 = 10$, $\lambda_3 = \lambda_4 = 6$) and annealing schedule. The Fig. 5 notebook builds the density and via matrices from the segment geometry and implements the layer-assignment Hamiltonian with $\lambda_D = 10$ and $\lambda_V = 4$. It also defines the optimality gap reported in Fig. 5f.

A figure is a claim, and the code that draws it contains the definitions the claim depends on: what is being plotted, against what reference, under what normalization. When those definitions sit in the plotting code, a replottable release is also a partially specified one, and a reader who has the instance, the objective, and the claimed outcome has everything needed to check the outcome without rerunning anything.

4 Verification Without the Machine

The routing instance

The paper describes the routing trajectory of Fig. 4d as converging to the ground state within 1,000 iterations, and Fig. 4e as showing the evolution “through suboptimal solutions to the optimal solution” [1]. In the notebook, the dashed line labelled “Ground state” is computed as `ground = np.min(H)`, the lowest Hamiltonian value along the plotted trajectory itself [2]. By construction, the trajectory reaches it.

An independent check is straightforward for an instance of this size. A rectilinear Steiner tree on these vertices is a spanning tree over the terminals and some subset of the candidate Steiner points, using only axis-aligned edges. Enumerating all 256 subsets of the eight candidates and computing a minimum spanning tree for each takes about a hundredth of a second.

Configuration	Tree length	Penalty terms	Notebook H
Released trajectory, iteration 1,000	36	all zero	36
Exhaustive minimum over Steiner subsets	32	all zero	32

Table 1: The routing instance defined in the Fig. 4 notebook. Both configurations are valid trees under every penalty term of the notebook’s Hamiltonian. The second is shorter by 4 units, or 12.5%. It also avoids the rectangular module outlines drawn in the figure.

The minimum tree connects the seven terminals through four Steiner points, and when it is encoded as edge and order spins and evaluated with the notebook’s own Hamiltonian function, every penalty term is zero and $H = 32$. The released trajectory ends at a valid tree of length 36, first reached at iteration 930, and that is the tree labelled optimal. The script used for these checks is short and is released alongside this essay [8].

There are explanations that would reconcile this with the paper. The notebook might define a slightly different instance from the one used for the hardware run, or the figure might have been meant to show a representative run rather than a successful one. The paper’s success statistics in Fig. 4f, which report optimal solutions in over 95% of runs beyond 10^4 iterations, cannot be checked either way, because the released spreadsheet records outcome frequencies but not the reference length against which “shortest” was judged. What the release does establish is that, for the instance and Hamiltonian it defines, the trajectory it contains does not reach the ground state, and the line labelled as one is the trajectory’s own minimum.

The layer-assignment instance

Fig. 5d is described as reaching the ground state within 1,000 iterations. Its notebook plots the second of the two released trajectories and again draws the “Ground state” line at the trajectory’s own minimum [2]. The released J and h reproduce the notebook’s Hamiltonian exactly up to an additive constant, which allows an independent search.

The second trajectory ends in a configuration from which every single spin flip raises the energy, a local minimum. The first trajectory, which the notebook uses for the layer diagram in Fig. 5e, ends 4.28 units lower. Two thousand restarts of a plain simulated annealing solver on the released J and h find no configuration lower than the first trajectory’s endpoint, which is therefore the best known state. The plotted trajectory stops about 0.5% above it on the

notebook’s energy scale. That is a small gap and well within what annealing is expected to leave. It is not the ground state, and the line says it is.

The optimality gap

Fig. 5f reports the optimality gap, defined in the paper as “the relative error between convergence value and minimum value,” falling below 2% beyond 10^4 iterations for problems of 20 to 100 segments [1]. In the notebook, the gap at each iteration count is computed relative to the minimum over the 100 trials at that same iteration count [2]. The reference therefore moves with the budget: at 10 iterations it is the best of 100 short runs, at 10^4 the best of 100 long ones.

A gap measured this way shows how tightly the trials cluster, not how far they are from the optimum. If all 100 runs converged to the same suboptimal state, the gap would be zero. The instances for sizes other than 40 are not released, so the distance to the true optimum cannot be checked for them. The notebook also defines an array of reference minima for the five sizes that the calculation does not use, and whose values for 80 and 100 segments lie above energies the released trials actually reach.

A second issue concerns the denominator. A relative error $|H - H_{\min}|/|H_{\min}|$ depends on where the zero of H lies, and the paper notes that the absolute value of this Hamiltonian is arbitrary: it becomes negative when λ_D greatly exceeds λ_V , which “does not affect the annealing process” [1]. The released J and h define the same landscape as the notebook up to a constant of about 392, and the same 4.28-unit shortfall is 0.5% of the best energy on one scale and 0.9% on the other. The Shape of Search observed that the height of such a landscape is not an observable of the search [7]. It is not an observable of the gap either, and a percentage computed against it inherits whatever constant the formulation happened to carry.

5 What Cannot Be Done

The checks above required no hardware and no machine code. They did not require the chip because none of them concerned the chip: they concerned instances, objectives, trajectories, and reference values. That is the sense in which the release is valuable beyond its stated purpose.

What cannot be done with it is rerun anything. The trajectories cannot be regenerated, the success statistics cannot be recomputed, and the outcome of a longer run or a different schedule cannot be seen. The obstacle is not mainly the chip. The paper itself shows that the chip’s functional role in the algorithm can be emulated: its methods describe a full-FPGA implementation in which shift registers, sigmoid lookup tables, and comparators replace the junctions, used to study device variation, and its shift-register comparison shows a 16-bit register matching the junctions’ success rate [1]. For claims about solution quality, the part that cannot be downloaded is also the part that can be replaced. The part that is needed and withheld is the code: the mapping, the schedule, the update loop, the evaluation criteria, and the instances. For claims about speed and energy, the chip is irreplaceable, and there the deposited switching and resistance data for Fig. 2 are the appropriate level of release. A reader cannot rerun a 40-femtojoule switch in software, and does not need to in order to check the probability curve.

The comparison axis is empty. The repository contains no data for the comparison table, in either its preprint or published form.

6 Custody

“Available on reasonable request” places the process under custody. The experiment remains reproducible in principle, and access to that reproducibility is at the discretion of the authors, mediated by a judgment of reasonableness that the requester does not make. Evidence on how such statements work in practice is not encouraging. A study of 1,792 articles whose data availability statements promised sharing found that 93% of authors either did not respond or declined, and only 6.8% provided the data [4]. A study of Nature-portfolio and PNAS papers with data available on request found that 14% of authors shared or indicated willingness to share [5]. These are studies of data rather than code, in other fields, and say nothing about the present authors. They show what kind of availability “on request” usually is.

Custody also changes who can check. A verification like the one above can be performed by anyone with the release and an afternoon. A verification that needs the code requires the requester to be judged reasonable, to wait, and to receive what was asked for. The checks that are easiest to perform are then the ones the release happened to enable, and the claims that depend on withheld material, such as the success statistics and the scaling curves, are the ones least likely to be checked at all.

7 A Counterpractice

The alternative is not complicated. It is to release the traversal rather than its record:

1. the problem instances, in the form the solver consumes and in the form a human can inspect;
2. the mapping from problem to Hamiltonian, as code;
3. the schedule and update loop, with a software model of the hardware sampler good enough to reproduce the algorithmic results;
4. the evaluation code, including how reference optima were obtained and how every figure of merit is computed;
5. the raw trajectories and outcome records;
6. for any comparison, the per-system figures, their sources, and the procedure that made them commensurable.

Released under a permissive or public-domain license, in version control, at the time the work is produced rather than after publication, such a release lets a reader rerun, verify, and compare without asking. It also protects the authors. A trajectory that stops above the ground state is an ordinary outcome of annealing and says nothing bad about the hardware. A figure that calls that point the ground state is a small error that a released solver check would have caught before submission.

A further practice follows from the findings here: any figure that labels a value as a ground state, an optimum, or a minimum should come with the method that certified it. A trajectory’s own minimum certifies nothing about the landscape. An exhaustive enumeration, an exact solver, or a stated best-known value from an independent method does.

8 Conclusion

The repository accompanying this paper was built to let readers regenerate three figures. It does that, and because its notebooks define the instances and objectives behind those figures, it does something more: it lets a reader check them. The checks find that the lines labelled ground state are trajectory minima; that for the released routing instance, a shorter valid tree exists than the one shown as optimal; that one plotted layer-assignment run ends at a local minimum; and that the optimality-gap figure measures dispersion among trials against a reference that depends on an arbitrary energy constant.

None of these findings touches the paper’s central contribution, the sub-nanosecond probabilistic switching of a voltage-controlled junction, which rests on device measurements that the repository also contains. All of them concern the algorithmic claims built on it, and those are exactly the claims whose full checking requires what was not released. The release is replottable, and by accident partly verifiable. It is not rerunnable, and the difference is where the questions it raises would be answered.

References

- [1] S. Li, Y. Zhang, A. Lee, Z. Zhu, L. Zeng, P. Wang, L. Gao, D. Wu, and W. Zhao. An Ising Machine for Combinatorial Optimization Based on Sub-Nanosecond CMOS-Integrated Magnetoresistive Random-Access Memory. *Nature Electronics*, 2026. doi:10.1038/s41928-026-01700-6.
- [2] SpinX-Lab. Voltage-controlled-Spintronic-Ising-Machine: source data and plotting scripts. GitHub repository, 2026. <https://github.com/SpinX-Lab/Voltage-controlled-Spintronic-Ising-Machine>. Accessed September 2026.
- [3] National Academies of Sciences, Engineering, and Medicine. Reproducibility and Replicability in Science. The National Academies Press, 2019.
- [4] M. Gabelica, R. Bojčić, and L. Puljak. Many Researchers Were Not Compliant with Their Published Data Sharing Statement: A Mixed-Methods Study. *Journal of Clinical Epidemiology*, 150:33–41, 2022.
- [5] C. Acciai, J. W. Schneider, and M. W. Nielsen. Estimating Social Bias in Data Sharing Behaviours: An Open Science Experiment. *Scientific Data*, 10, 2023. doi:10.1038/s41597-023-02129-8.
- [6] Flyxion. The Comparison Table: Boundaries, Estimates, and the Revision of a Benchmark. September 2026.
- [7] Flyxion. The Shape of Search: Reachability, Measurement, and Attraction in Physical, Cognitive, and Conversational Systems. September 2026.
- [8] Flyxion. `verify_vsim_release.py`: checks on the released VSIM figure data. Public domain, September 2026.